

1- JSON

- **JSON = JavaScript Object Notation**
- **plain text** format for sharing data between server and client
- similar to the syntax for creating JavaScript **objects** .
- JSON data can be written in any programming language.

- ال **json** هي طريقه او format لكتابه الداتا مستوحاه من شكل ال object الموجود ف javascript لانه علي شكل **key-value** ولكن هي **مش لغه برمجيه**.
- طب يعني اي **plain text format** يعني مجرد نص عادي اي برنامج او اي لغه تقدر تقرأه يعني لو فتحت اي ملف ال extension بتاعته **json**. هنلاقي كلام عادي مش binary او ارقام غريبه يعني حتي لو فتحت ب notepad هيفتح عادي

```
//user.json
{
  "username": "admin",
  "active": true
}
السيرفر بيرجع جيسون ف البراوزر هيستقبل الجيسون ويعرضه ف الصفحه
```

- بيبقي وظيفه ال json الحقيقيه انه :
- السيرفر بيعت داتا للـ Frontend
- أو الـ Frontend بيعت داتا للسيرفر

Note:

Every JSON looks like an object, but not every object is JSON.

يعني json مستوحاه من js objects ولكن json بتختلف ان :

- ال **key** لازم يتحط بين double quotes ولو حطيته بين **' '** هيطلع ايرور لازم **""**
- مينفعش فيه **function**
- مفيش فيه **undefined** خالص

why we use **json** rather than **xml**

Feature	JSON	XML
Format	text format	Markup language
Syntax	Based on key-value pairs	Based on tags
Readability	Easy to read and write	More complex and verbose

Feature	JSON	XML
Data Size	Smaller	Larger
Parsing Speed	Faster	Slower
Data Structure	Objects and Arrays	Tree structure

• JSON

- عبارته عن text بسيط وخفيف فهو اخف واسهل ف الاستخدام
- يعتمد علي ال key - value
- ال structure بتاعه عبارته عن array او object
- الانسان بي فهمه بسرعه لان شكله قريب من js (سهل ف ال read و write)
- ال data size بتاعته صغيره (حجم أقل = تحميل أسرع من السيرفر)
- تحويل الداتا لشكل يفهمه البرنامج بسرعه جدا (Parsing Speed)
- أغلب ال APIs بترجع JSON

• XML (eXtensible Markup Language)

- فهو Markup Language يعتمد علي tags كتير زي HTML
- شكله أطول وأعقد حتى لو الداتا بسيطة بسبب ال tags
- فيه tags كتير ف القراءة بتاخذ وقت أكثر
- حجمه أكبر بسبب التكرار في tags
- تحويل الداتا لشكل يفهمه البرنامج بطيء جدا لانه اعقد ودا بيأثر علي ال performance.
- يعتمد علي Tree structure (يعني فيه root و children) ودا بيبقي صعب التعامل معاه علي عكس لو الداتا علي شكل lists و api

```
<user>
  <name>Tahany</name>
  <age>21</age>
  <skills>
    <skill>C++</skill>
    <skill>JavaScript</skill>
    <skill>Linux</skill>
  </skills>
</user>
parent كل حاجه جوا تاج وكل عنصر ليه
parsing ودا بيبقي اصعب ف عمليه ال
```

```
{
  "name": "Tahany",
  "age": 21,
  "skills": ["C++", "JavaScript", "Linux"],
```

JSON

```

    "address":{
      "egypt":"","
      "germany":[1,2,3]
    }
  }
  console.log(ob.address.germany[0]);

```

- json syntax:

ممکن نخط فیہا string / array / object / number / boolean / null
 ممکن نخطہم ف نفس ال line ولكن الافضل نکتب زي فوق کدا عشان بتکون more readable

• JSON converting:

بتبقى علي هيئه json لما بيرجع الداتا علي شكل api يعني عبارہ عن نص فال plain text عبارہ عن json احنا قولنا ان ال عشان اقدر انا اكسيس علي الداتا دي ف البراوزر عندي object عشان کدا انا بحتاج احوالها ل string

-> عشان کدا هنستخدم parse / stringify

-> parse لو الداتا جاي من السيرفر للفرونت هستخدم

-> string عشان احوالها ل stringify اما لو رايحه من الفرونت للباك هستخدم

```

//from server to client
let jsonText = '{"name":"Tahany","age":21,"isStudent":true}';
let user = JSON.parse(jsonText);
console.log(jsonText);
console.log(typeof jsonText);//string
console.log(user)
console.log(typeof user)//object

//from client to server
let jsonText = {"name":"Tahany","age":21,"isStudent":true};
let user = JSON.stringify(jsonText);
console.log(jsonText);
console.log(typeof jsonText);//object
console.log(user);
console.log(typeof user);//string

```

JSON



2- API

API (Application Programming Interface)

- هو ببساطه عبارته عن الوسيط بين ال client و server يعني اول م اليوزر يعمل request عشان يجيب بياناته مثلا من السيرفر، هو مش بيكلم الداتا بيز مباشرة، هو بيكلم API وال API هو ال بيكلم الداتا بيز ويجيب منها الداتا دي
- ليه مينفعش استخدم ال server علطول عشان الامان مينفعش اسيب الداتا بيز اي حد يعرف ي access عليها

٣

3- Asynchronous vs Synchronous Programming

- *Asynchronous* :
 - Operations run in parallel
 - This means that another operation can occur while another one is still being processed
- *Synchronous* :
 - Operations run in sequence
 - each operations must wait for the previous one to complete
- يعني ال *Synchronous* العمليات بتننفذ بشكل متسلسل ورا بعضها ومينفعش واحده بتننفذ الا لما ال قبلها تخلص ال كانت بتعمله.
- علي عكس ال *Asynchronous* العمليات بتشتغل في شكل parallel يعني لو عمليه هتاخذ وقت طويل هسيبها بتننفذ واروح انفذ ال بعدها كمان لحد اما ال هتاخذ وقت طويل تخلص ف ارجلها

JAVASCRIPT

```
//synchronous
console.log(1);
console.log(2)
window.alert("alert");
console.log(3);

//asynchronous
console.log(1);
console.log(2)
setTimeout(()=>{
  console.log("operations");
},1000)
console.log(3);
```

ليه JavaScript ما تنفذش `setTimeout` ؟

- لان JavaScript نفسها ما بتعرفش تعدّ وقت ولا تستنى عشان كده ما تقدرش تنفذ `setTimeout` لوحدها ف بتسلمها لل **Browser (Web APIs)** وهو ال بينفذها
- ولان JavaScript لغة **Single-Threaded** يعني اي؟
 - يعني بتنفذ تاسك واحد ف نفس الوقت يعني لو هي ال كانت بتنفذ `setTimeout` كانت الصفحة كلها هتقف ومش هيتنفيذ اي حاجه لحد اما عدد الثواني يخلص ودا مينفعش
 - **هل د يعتبر عيب ؟** لا لان البراوزر عباره عن `ui / scroll / click / animation` لو js قعدت تنفذ كل حاجه ف نفس الوقت وتسبب تاسك وتنفذ ال بعده كدا الصفحة هتعلق وكل حاجه هتبقى ابطأ ويتاخذ وقت
- اذن **JavaScript** معندوش timer معندوش ساعه ومبيعرفش يستنى وقت وينتظر فعشان كدا `setTimeout` مش جزء من **JavaScript Engine**
- واحد JS Engine = Thread
- كثير Browser = Threads

طيب كل الكلام ال بيحصل ف ال Asynchronous بيتم ازاي ؟ وليه اي حاجه البراوزر بينفذها بتبقى ف الآخر ؟

ف 4 حجات هنتكلم عليهم بيحققوا مبدا ال Asynchronous :

- ****Call Stack (stack trace) :**
 - When Function Called It Added To The Stack
 - When Function Executed It Removed From The Stack
 - After Function-Is Finished Executing The Interpreter Continue From The Last Point
 - Work Using **LIFO** Principle => Last In First Out
 - Code Execution " Is **Synchronous** .
 - Call Stack Detect Web API Methods And Leave It To The Browser To Handle It
- يعني ال call stack هو **المكان** ال بيحصل فيه execution لل function بعد اما يتعمل ليها `call`.
- ولو عندنا فانكشن بتعمل `call` لفانكشن تانيه جواها والتانيه دي بتعمل `call` لفانكشن تالته هيتنفيذوا بمبدأ ال **LIFO** (**last in first out**) يعني اخر فانكشن هتقابلني هي اول واحد هتتنفذ.
- والفانكشن دي بتننفيذ `synchronous` يعني مينفعش اروح للتانيه غير لما الاول تخلص.
- ال call stack بيتعرف علي ال `Methods API` ويبسيبها في الآخر لل browser عشان ينفذها.

JAVASCRIPT

```
function one (){
  console.log("one");
}
function two(){
  one();
  console.log("Two")
}
```

```

}

function three(){
  two();
  console.log("three");
}

three();

```

- **Web API**

- Methods Available From The Environment -> Browser

- يعني هو دا البراوزر ال بينفذ كل الحجات ال js مش هتعرف تنفذها زي `setTimeout` بس كل الحجات دي مش هيحصلها `call` وتنفذ الا لما ال `call stack` يخلص كل الفانكشن ال عنده
 - طيب هو دلوقتي هيعرف منين ان ال `call stack` خلص عشان ينفذ الحجات ال js مبتعرفش تنفذها ؟ وكمان لو عندي مثلا اكتر من فانكشن `setTimeout` هينفذ مين الاول او هيحطهم فين لحد اما `call stack` يخلص ؟ <= دول هيكونوا الحاجتين ال ناقصين

```

setTimeout(()=>{
  console.log("Web Api");
},0)
function one (){
  console.log("one");
}
setTimeout(()=>{
  console.log("Web Api 2");
},0)
function two(){
  one();
  console.log("Two")
}

function three(){
  two();
  console.log("three");
}

three();

```

JAVASCRIPT

- **Event loop and Callback queue**

- دلوقتي احنا قولنا ان ال `call stack` بينفذ كل ال `calls` بتاعت ال `functions` بمبدأ `lifo` ولو في اي فانكشن مبيعرفش ينفذها ببيعتهها لل `web api`
 - بس لو احنا عندنا اكتر من فانكشن ال js مبيعرفش ينفذهم ومضطرين يستنوا لحد اما ال `call stack` يخلص عشان يتنفذوا ؟

- يبقى كذا هيستتوا ف ال **callback queue**
- بس احنا هنعرف منين ان ال **call stack** خلص الفانكشن بتاعت js ؟
- عن طريق ال **event loop** بيعمل **check** علطول علي ال **call stack** ويستني لحد اما يبقى فاضي وبعدين بيعت كل الفانكشن ال موجوده في **callback queue** وبيعتهم لل **call stack** بس بيعتهم ب **FIFO(first in first out)** يعني ال اتضاف اولاف ال **queue** يتحذف اولاف

"Pasted image 20260202021337.png" could not be found.

"Pasted image 20260202020752.png" could not be found.

♦ AJAX vs JSON

Feature	AJAX	JSON
What it is	Technique / Concept	Data format
Purpose	Send & receive data asynchronously	Represent and store data
Role	How data is sent	What the data looks like
Depends on	JavaScript + Browser APIs	Plain text
Can work without the other?	Yes	Yes
Common usage	Fetch data without page reload	Exchange data between systems

- يعني ببساطه **AJAX** مش داتا هو طريقة او أسلوب بنستخدمه عشان :
 - نبعث request للسيرفر
 - نستقبل response
 - من غير ما نعمل Reload للصفحة (يعني كنت الاول كل اما ابعت request للسيرفر الصفحة كلها يتعمل ليها reload كلها ودا كان ببدايق اليوزر وياخد وقت طويل **ajax** جت بقا عشان تحل دا)
 - يعني: **AJAX** = طريقة التواصل
 - انما **JSON** هو شكل الداتا نفسها
 - يعني:
 - السيرفر بيرجع داتا
 - الداتا دي مكتوبة بصيغة **JSON**
 - **JSON** = شكل الداتا
 - -> يعني **AJAX** بيستخدم **JSON** (لان هو بيعت ريكويست فالريكويست دا الداتا ال فيه بتكون علي شكل json)

JAVASCRIPT

```
fetch("https://api.example.com/user")
```

```

.then(res => res.json())
.then(data => {
  console.log(data.name);
});

{
  "name": "Tahany",
  "age": 21
}

```



Callback Hell Or Pyramid Of Doom

- **callback** : A function that is passed into another one as an argument to be executed later.
- يعني ال callback هي عبارته عن فانكشن بنمررها لفانكشن تانيه ك argument ليها عشان يتم تنفيذها بعد كدا
- طيب بيحصل امتي ال **callback hell** لما يكون فانكشن جواها فانكشن ودي كمان جواها فانكشن وكلهم معتمدين علي بعض وخصوصا لو كانت الفانكشن دي من ال js مبتعرفش تنفذهم ولازم البراوزر ينفذهم زي **setTimeout()** ف كدا هيبقي عندي كميه فانكشن مستنيه تنفذ ومتاخره

JAVASCRIPT

```

setTimeout(()=>{
  console.log("result 1");
  setTimeout(()=>{
    console.log("result two");
    setTimeout(()=>{
      console.log("Result three");
      setTimeout(()=>{
        console.log("Result four");
      },1000)
    },1000)
  },1000)
},1000);

```

```

delayLog("Result one", 1000, () => {
  delayLog("Result two", 1000, () => {
    delayLog("Result three", 1000, () => {

```

```

    delayLog("Result four", 1000, () => {
      console.log("Done");
    });
  });
});
});
});

```

Promise :

- Promise in JavaScript is like promise in real world.
- Promise is something that will happen in the future.
- promise avoid callback hell.
- promise is the object that represent the status of asynchronous operations and its resulting value .
- ال promise بمعنى وعد في ال js زي العالم الحقيقي بالظبط يعني وعد ان الحاجه دي هتحصل ف المستقبل وهي جت بقا عشان تحل مشكله ال callback hell لان js مينفعش يكون ف فانكشن هتاخد وقت بتتنفذ جواها وتسببها وتمشي.
- وكمان ال promise هو ال object ال بيمثل حاله ال asynchronous operations والنتائج ال هترجع منها

Promise status:

- 1- Pending : initial state
- 2- fulfilled : completed successfully
- 3- Rejected : failed

- في 3 حالات لل promise وهما :
 - حاله ال pending وهي الحاله ال بنبدأ بيها ومبيظهرش اي نتيجته تدل ان اي حاجه اتحققت او لا
 - حاله ال fulfilled بتدل ان ال promise اتحقق وكل حاجه تمت بنجاح
 - حاله ال rejected بتدل ان ال promise متحققش او العمليه فشلت

-> The Promise object supports two **properties** : state and result.

- While a Promise object is " **pending** " (working), the result is **undefined** .
- When a Promise object is " **fulfilled** ", the result is a **value** .
- When a Promise object is " **rejected** ", the result is an **error object** .

-> **Promise Syntax:**

```

let promise = new Promise((resolve, reject) => {
  // async operation

```

JAVASCRIPT

```
});

let p = new Promise((resolve, reject) => {
  resolve(5);
});

p.then(result => {
  console.log(result);
});
```

انا هنا عندي كلاس اسمه promise هعمل منه object وهي بتاخذ 2 parameters

- resolve : هي فانكشن بتناديها لما العملية تنجح وبترجع النتيجة
- reject : فانكشن بتناديها لما العملية تفشل

JAVASCRIPT

```
function delayLog(message,time){
  let prom=new Promise((resolve,reject)=>{
    setTimeout(()=>{
      console.log(message);
      resolve();
    },time);
  });
  return prom;
}

delayLog("Result one" , 1000)
  .then(()=>delayLog("result two",1000))
  .then(()=>delayLog("result three",1000))
  .then(()=>delayLog("result four",1000))
  .catch(err=> console.log(err));
```

-> `Promise.then()` takes **two** arguments, a **callback** for success and another for failure. Both are **optional** , so you can add a callback for success or failure only.

- طيب انا ال `.then()` دا بيتنفذ لما ال promise يتحقق ويبقى fulfilled وبتستقبل القيمة ال اتبعثت من ال resolve وهي بتاخذ اثنين **callbacks** برودو عشان واحد لو اتحقق وواحد لو متحققش
- بالنسبة لل `.catch()` بيتنفذ لما ال promise تبقي **rejected** .
- كل **then** بتبقي عبارة عن promise لل **then** ال بعدها ف بتشكل **Promise Chaining**

```
// هنا عشان ارجب ال reject
function delayLog(message,time){
  let prom=new Promise((resolve,reject)=>{
    setTimeout(()=>{
      if(isSuccess){
        console.log(message);
        resolve();
      }else{
        reject("error at :"+message);
      }
    },time);
  });
  return prom;
}

delayLog("Result one" , 1000,true)
.then(()=>delayLog("result two",1000,true))
.then(()=>delayLog("result three",1000,true))
.then(()=>delayLog("result four",1000,true))
.catch(err=> console.log(err));
```

Async and Await

ال `async / await` مبيلغوش Promise هما مبنيين عليهم يعني أي function مكتوب قبلها `async` بترجع **Promise تلقائي** فمش محتاجين نكتب `new Promise` بإيدنا يعني هما جايين عشان يكتبوا كود ال promise بشكل clean أكثر .

```
//with promise
function getData() {
  let users = ["fady"];
  return new Promise((res,rej)=>{
    if (users.length > 0) {
      res("user found");
    } else {
      rej("euse not found");
    }
  });
}

//with promise
function getData() {
  let users = ["fady"];
```

```

    if (users.length > 0) {
        return Promise.resolve("users found");
    } else {
        return Promise.reject("not found");
    }
}

//with async
async function getData(){
    let users = ["fady"];
    if (users.length > 0) {
        return "user found";
    } else {
        return "not found";
    }
}

console.log(getData());
getData().then(
    (resolvedValue)=> console.log(resolvedValue),
    (rejectedValue)=> console.log(rejectedValue)
);

```

- دي `return Promise.resolve("users found")` عباره عن method جاهزه معناها رجعلي promise جاهزه ال state بتاعتها fulfilled

```

//without promise
function wait(){
    setTimeout(()=>{
        console.log("done waiting");
    },2000);
}

function wait(){
    return new Promise((res)=>{
        setTimeout(()=>{
            console.log("done waiting");
        },2000);
    })
}

//لان محدش مستني النتيجة res هنا مستدعيتش ال

```

JAVASCRIPT

```
console.log("start");
  wait();
console.log("end");

function wait(){
  return new Promise((res)=>{
    setTimeout(()=>{
      console.log("done waiting");
      res("finished");
    },2000);
  })
}
```

```
async function test() {
  console.log("start");
  let result=await wait();
  console.log(result);
  console.log("end");
}
```

```
test();
```

```
function wait(){
  return new Promise((res)=>{
    setTimeout(()=>{
      console.log("done waiting");
      res("finished");
    },2000);
  })
}
```

```
console.log("start");
```

```
wait().then(result => {
  console.log(result);
  console.log("end");
});
```

`await` لازم تكون جوه `async function`

- لو استخدمت `then` هي نفس النتيجة نفس كل حاجه الفرق بس ف طريقه التفكير والكتابه

Promises	async / await
<code>.then()</code>	<code>await</code>
<code>.catch()</code>	<code>try / catch</code>
chaining	linear code

practical

JAVASCRIPT

```
function getUser(callback) {
  setTimeout(() => {
    const user = { id: 1, name: "Zeyad" };
    console.log("User loaded");
    callback(user);
  }, 1000);
}

function getPosts(userId, callback) {
  setTimeout(() => {
    const posts = ["Post 1", "Post 2"];
    console.log("Posts loaded");
    callback(posts);
  }, 1000);
}

function getComments(post, callback) {
  setTimeout(() => {
    const comments = ["Nice", "Great"];
    console.log("Comments loaded");
    callback(comments);
  }, 1000);
}
```

JAVASCRIPT

```
getUser(user => {
  getPosts(user.id, posts => {
    getComments(posts[0], comments => {
      console.log("Final Result:", comments);
    });
  });
});
```

```
});
```

Solution

JAVASCRIPT

```
function getUser() {
  return new Promise(resolve => {
    setTimeout(() => {
      const user = { id: 1, name: "Zeyad" };
      console.log("User loaded");
      resolve(user);
    }, 1000);
  });
}

function getPosts(userId) {
  return new Promise(resolve => {
    setTimeout(() => {
      const posts = ["Post 1", "Post 2"];
      console.log("Posts loaded");
      resolve(posts);
    }, 1000);
  });
}

function getComments(post) {
  return new Promise(resolve => {
    setTimeout(() => {
      const comments = ["Nice", "Great"];
      console.log("Comments loaded");
      resolve(comments);
    }, 1000);
  });
}
```

JAVASCRIPT

```
getUser()
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0]))
  .then(comments => console.log("Final Result:", comments))
```

```
.catch(err => console.log(err));
```